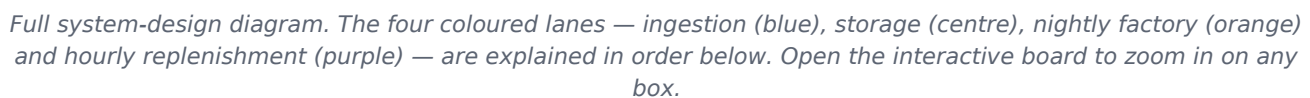


Batch hybrid GraphRAG (vector + knowledge graph) · generation-side, with zero runtime consumers. Scope locked 2026-07-17: a nightly run plus an hourly top-up, and the chatbot does **not** consume RAG.

<https://excalidraw.com/#json=tHfWlVQu3MQjAviTWGZ4y,nakbyPaaseSsjDy8UUYM5w>



How it works

This isn't the usual "chat with a PDF" kind of RAG, where you ask a question and the AI answers you on the spot. There's nobody waiting on the other end. Instead, a scheduled background job works through the six books and the topic map on its own, uses AI to write fresh practice questions, checks them, and files them away. By the time a student sees anything, it has already been generated and verified — they only ever meet saved, pre-checked content. The "hybrid" label just means the system keeps two kinds of memory: one holds what the six books actually say (the vector side), and the other holds how the topics relate to each other (the graph side).

Chunking — cutting the books into pieces

A chunk is a small piece of a book the computer can search. We don't chop every 500 words and call it done; we cut where a teaching unit naturally ends. One explanation, one worked example, or one problem becomes one chunk — because splitting a formula or a solution down the middle leaves you with garbage. Every chunk carries its book and page number with it, plus a fingerprint (the `content_hash`) so duplicates get thrown away. One thing worth flagging up front: the 4,943 chunks we already have came in with messy type labels, so cleaning those up is part of the build.

Embedding — turning text into searchable numbers

Embedding takes a chunk and turns it into a list of 1,536 numbers that stands in for what the chunk is about. The useful property is that two chunks covering the same idea land on similar numbers, which is exactly what lets us search by meaning instead of hunting for keywords. We run this through OpenAI's text-embedding-3-small model and keep the numbers right inside the Postgres database using the `pgvector` extension — so there's no separate vector database to babysit. Each chunk is embedded once, when it comes in, and a checkpoint file means a run that dies halfway can pick up where it left off. Nothing here ever fires because a student did something, so at runtime it costs nothing. There's one cleanup job in this area too: the ontology registry's embeddings are all sitting at zero because they were never computed, so we'll regenerate them.

Retrieval — finding the right pieces

Retrieval happens in two steps. First it narrows down by topic using the map — so if we're generating Nikhilam questions, only Nikhilam-related chunks even get considered. Then it ranks whatever's left by meaning, using cosine similarity, keeps the best five, and throws out anything scoring below 0.70. For formulas, where meaning-based search tends to fall flat, there's a plain keyword fallback running on `pg_trgm`. All of this happens in exactly three spots, and every one of them lives inside the factory — never in front of a user.

Lane 1 · Ingestion Pipeline

Raw book to searchable knowledge — runs only when you add a new book.

The 4,943 chunks already sitting in the system are what this lane produced last time, which is why the shelves are stocked from day one. The sources are the six math books, as PDFs. The **chunker** (station 0) goes first, cutting each book into teaching-unit pieces, tagging what type each piece is, and holding on to the page numbers. Next the **digitizer** (station 1) turns the PDF into plain text or markdown — a necessary step, because LangExtract two stations later can't read PDFs. Station 2

handles **context and embedding** together: it computes each piece's 1,536-number vector, looks up its nearest existing neighbours so the next station has something to work from, and cleans up any odd Unicode with NFKC normalization.

Station 3 is **LangExtract**, and it's the AI reading step (the red one). Its job is to pull structured facts out of the text — skills, techniques, traps, examples — under one hard rule: every fact has to point back to the exact characters in the source that support it. If it can't point to them, the fact gets thrown out. That's the built-in guard against hallucination, and we're piloting it on a single Tirthaji chapter before leaning on it for everything. After that, station 4 runs a **math audit** — SymPy recomputes every answer, with a five-second ceiling per check; ten timeouts in a row trips a circuit breaker and stops the run. Finally, station 5, **dual-DB ingest**, writes everything into both stores at once: text and vectors go to Postgres, entities and relationships go to Neo4j. It uses MERGE ("update it if it's already there"), so you can safely re-run the whole thing without making a mess.

Lane 2 · Storage

The pantry, the map, the warm tray, and the plug — a dual store that's never collapsed into one.

- **PostgreSQL + pgvector (the Ledger)** — the record book. It holds the 4,943 chunks and their vectors, every problem we generate (with fingerprint-based dedup), the trust score attached to each problem, an audit log of every single AI call, the replenishment history, and the reference-library pages.
- **Neo4j (the GPS)** — the topic map. Every bubble is one of a handful of things: a Skill (467) is one learnable ability, like base-100 Nikhilam multiplication; then there are 807 Problems, 791 SolveAlongs (guided walkthroughs), 590 Explainers (lesson cards), 908 Traps (known common mistakes), the 16 Sutras (the Vedic formulas), and the 6 Books. The arrows are where the meaning lives: a Sutra shows up as Skills (MANIFESTS_AS); Skills point at other Skills to set the learning order (FRONTIER_OF, NEXT_TOPIC, REQUIRES); a Skill has to come before a Problem (PREREQUISITE_OF); SolveAlongs teach Skills and Explainers explain them, both pointing back to the Book they came from; and Problems and SolveAlongs point to the Traps a student is likely to hit. One thing to watch: the old schema file still refers to :Technique nodes, but the actual database uses :Skill — always code against the real one.
- **Redis (the warm tray)** — the questions cooked overnight sit here for 24 hours so the app can grab them instantly all day. It also keeps the "recently seen" fingerprint list and the counters tracking AI spend.
- **Integration surface (the plug)** — deliberately thin — just three wires. First, the two Celery task names the main repo already schedules, where we're really just filling in empty stubs. Second, the Postgres tables we write to. Third, the Redis tray the app reads from. Underneath it all sits the single most important rule: game-loop isolation. While a student is actually practising, none of this machinery is touched — no AI, no retrieval, completely offline. And per your call, the chatbot stays out of RAG too.

Lane 3 · Nightly Batch Factory

The once-a-day RAG hit, at 00:00 UTC.

The trigger is the alarm clock (Celery beat), and it comes with a hard token budget so a night's run physically can't overspend. Then the **pool composer** asks the obvious question — which subtopics are running low on fresh questions? — checking the map along with the “already seen” and “already mastered” lists so nobody gets served repeats. Generation runs at two tiers and dodges AI wherever it can: **T1** just pulls from the 1,505 ready-made templates, while **T2** takes a pattern and rolls fresh numbers into it — seeded random, so it stays reproducible — turning out 100 per pattern, each one re-checked against plain arithmetic the moment it's made.

Everything then goes through the **7-stage auditor**, the quality-control line: all fields present, the LaTeX actually renders, the math holds up under SymPy, the domain rules are satisfied (for instance, whether the chosen base is even valid), the declared traps make sense, it isn't a duplicate, and finally 2 of 3 AI judges sign off — that last one being the only place AI shows up in this lane. Passing the audit still isn't the finish line. Every new question has to climb a **trust ladder**: it starts QUARANTINED, moves to SANDBOX (shown at most 50 times, never counting toward mastery scores or mock exams), then TRUSTED, and finally LIVE. If three questions from the same pattern end up quarantined, the whole pattern gets frozen. Once that's done, **write-out** saves everything to Postgres, loads the Redis tray for the day, and files a run report covering tokens spent, pass rate, and pool depth.

Lane 4 · Hourly Replenishment

The top-up cook.

Every hour it asks whether any subtopic's unused pool has dropped below 50, and if so, it makes 100 more — always reaching for the cheapest method first and only escalating when something fails. It opens with **T1 resample** (no AI): the same patterns, but with wider number variation and stricter dedup. If that's not enough, **T2 escalate** (still no AI) generates at the next difficulty level. Then **T3 bridge** (no AI again) borrows from a prerequisite or sibling subtopic, but never more than one hop away on the map.

Only if T1 through T3 turn up nothing does **T4 LLM regenerate** kick in — the rare, red safety valve — where the AI writes new template variants, grounded in map context plus the top five book chunks, then runs the gauntlet of SymPy and the jury. Last in line is **T5 convert** (no AI), which turns explainer examples into practice questions by hiding the steps and shuffling the numbers; if the system ever has to reach all the way down to T5, that fires an alert to you. And if literally everything fails, it **degrades gracefully** — widening the topic, dropping into review mode, showing a mastery summary, and ending the session politely. One detail here matters a lot: the dashed red arrow looping back up means everything this lane produces goes right back through the 7-stage auditor. There is no side door into the question pool.